

Fault-Tolerance Scaffold Simulator

Hardware Errors, Measurement Errors, and Software Decoding

Boris Kiefer
 New Mexico State University
bkiefer@nmsu.edu
[GitHub](#)
[LinkedIn](#)



Anchor: Classical repetition-code scaffold for introducing the systems logic of quantum error correction.

Purpose: Demystify fault tolerance as a hardware–measurement–software co-design problem before introducing full quantum error-correction machinery.

Abstract

Fault-tolerant quantum computing is often introduced through specialized language: logical qubits, syndrome measurements, stabilizer codes, decoders, thresholds, and physical error rates. This brief presents a deliberately classical simulator that exposes the core systems idea before introducing quantum mechanics. A logical bit is encoded into an odd-length string of physical bits. Each correction cycle introduces fresh hardware errors with probability p , measurement errors corrupt the software-visible readout with probability q , and a majority-vote decoder attempts to recover the logical value. Running multiple correction cycles models the sustained error–correction loop that fault tolerance requires in practice. The simulator illustrates four central lessons: redundancy plus software decoding can suppress logical failure below any target rate, provided hardware errors are below threshold; $p = 0.5$ marks a total-randomness boundary where majority voting loses all information, but practical operation is governed by much stricter logical-error targets such as 10^{-2} , 10^{-3} , or lower; sustaining logical fidelity over many correction cycles requires the per-cycle error rate to stay below threshold; and a resource-estimation curve directly answers how many physical bits are needed to reach a target logical error rate for a given hardware error p — the key question for quantum hardware roadmap planning.

Core message. Fault tolerance is not hardware alone and not software alone. It is the co-design of hardware error rates, measurement fidelity, decoder logic, corrective action, and cycle budget.

Introduction

Quantum computing is moving toward a fault-tolerance-centered narrative. A useful quantum computer is not built from perfect physical qubits. Instead, reliable logical behavior must emerge from imperfect hardware, repeated measurements, classical decoding, and corrective control. The threshold theorem formalizes this idea: under suitable assumptions, error correction can suppress logical errors when physical error rates are below an appropriate threshold [1]. Stabilizer codes and surface-code architectures provide practical frameworks for this goal [2, 3].

The current state of the field reinforces this systems view. Recent surface-code experiments have shown logical-error suppression through code scaling and below-threshold operation with real-time decoding [7, 8]. Industry roadmaps and architecture papers increasingly frame progress around logical qubits, real-time decoding, compiler/runtime integration, micro-architecture, and full-stack fault tolerance rather than only physical-qubit counts [10, 11, 9].

However, fault tolerance can appear opaque to students, researchers in adjacent fields, and technical stakeholders. The present simulator therefore starts from a simpler classical analogue: repeated bits and majority voting. This demystifies the systems logic behind fault tolerance while avoiding the complications of quantum measurement, no-cloning, phase errors, leakage, and syndrome extraction.

Theory

Classical repetition scaffold

A logical bit $b \in \{0, 1\}$ is encoded into an odd-length string of N physical bits,

$$b \longrightarrow \underbrace{bb \cdots b}_{N \text{ bits}}, \quad N = 1, 3, 5, \dots \quad (1)$$

Each physical bit is independently flipped by the hardware with probability p . Each measured bit is then independently corrupted by readout/measurement error with probability q :

logical bit $\rightarrow$ hardware error $\rightarrow$ measurement error $\rightarrow$ software decoder.

Under independent symmetric errors, the probability that the software observes a wrong bit is

$$r = p(1 - q) + (1 - p)q = p + q - 2pq. \quad (2)$$

The majority-vote decoder fails when more than half of the observed bits are wrong. Therefore, for odd N ,

$$p_L(N, p, q) = \sum_{k=(N+1)/2}^N \binom{N}{k} r^k (1 - r)^{N-k}, \quad (3)$$

where p_L is the logical failure probability per correction cycle.

For $N = 1$ there is no redundancy and no majority protection. With $q = 0$, Eq. (3) gives $p_L = p$ exactly, so the $N = 1$ curve lies on the $p_L = p$ reference line when measurement is ideal.

Multi-cycle error correction

A single correction cycle is not enough to model sustained logical operation. Real fault-tolerant systems run many error-correction cycles continuously, and fresh hardware errors are introduced during each cycle.

The simulator models this by applying independent hardware errors with probability p and measurement errors with probability q at the start of every cycle, followed by majority-vote correction. The corrected physical state is carried forward into the next cycle, so each cycle both introduces new errors and attempts to repair the accumulated damage.

When the per-cycle logical failure rate p_L is small (below-threshold operation), successive correction cycles are approximately independent Bernoulli trials. The logical failure probability after k cycles is therefore approximated by the recurrence

$$P_{\text{fail}}(k) = 1 - (1 - p_L)^k, \quad (4)$$

where p_L is the single-cycle rate from Eq. (3). This expression is exact when each cycle is truly independent and slightly overestimates failure when p_L is large. Equation (4) captures the key practical constraint: even a small per-cycle logical failure rate accumulates over many cycles, so useful quantum computation requires p_L to be small enough that the total failure budget across all cycles remains acceptable.

Sustained operation requires sustained correction. Fault tolerance is not a one-time fix. It is a continuous loop: introduce errors, measure, decode, correct, repeat. The per-cycle error rate must stay below threshold for every cycle in the computation.

Information limit versus useful operating regime

For the idealized independent-error model with $q < 0.5$, the majority-vote information boundary occurs at $p = 0.5$. At this point each physical bit is random relative to the original encoded bit; majority voting has no usable information and is equivalent to guessing.

This boundary is important pedagogically, but it is not the practical engineering target. Useful operation requires logical failure rates far below the information limit. The simulator therefore distinguishes three conditions:

- **Information boundary:** $p = 0.5$, where majority voting loses information.
- **Improvement boundary:** $p_L = p$, where encoding and decoding are no better than using one physical bit.
- **Application target:** $p_L < p_{\text{target}}$, such as 10^{-2} , 10^{-3} , or lower depending on workload and cycle count.

Resource estimation

A natural engineering question follows from the multi-cycle picture: given a hardware error rate p , a measurement error rate q , and a cycle budget k , what is the minimum code length N needed to keep the total logical failure probability below a target p_{target} ?

Using Eq. (4), the required N is the smallest odd integer satisfying

$$1 - (1 - p_L(N, p, q))^k \leq p_{\text{target}}. \quad (5)$$

The simulator sweeps N analytically and plots p_L versus N for multiple hardware error values on a logarithmic scale, with horizontal reference lines marking selected logical-error targets. This resource-estimation curve directly translates hardware error rates into physical-bit overhead — the central quantity in quantum hardware roadmap planning.

A qualitative connection to recent fault-tolerance architectures is instructive. Proposals such as the IonQ Walking Cat architecture [9] are organized around exactly this resource question: how many physical operations and what level of hardware fidelity are needed to reach a target logical error rate over a meaningful circuit depth? The classical scaffold answers the same question with majority vote in place of a quantum code, making the resource tradeoff visible without quantum machinery.

Simulator Controls

The simulator is intentionally small. Its controls expose the minimal hardware–software trade space across three panels.

Control	Meaning	Role in the simulator
N	String length	Redundancy level; restricted to odd values.
p	Hardware bit-flip probability per cycle	Physical error channel before measurement.
q	Measurement/readout error probability per cycle	Corrupts the evidence seen by the decoder each cycle.
k	Correction cycles	Number of sustained error–correction rounds. Panel (a) and (c) update for the selected k .
trials	Monte Carlo samples	Compares sampling to the analytical result.
$p_{\max}, q_{\max}$	Heatmap axes	Zoom into the useful lower-error operating region.
p_{target}	Logical-error contours	User-selected performance goals shown in all panels.
$N_{\max}, p$ values	Resource panel range	Code lengths and hardware error rates shown in panel (c).

Panel (a) shows the full conceptual threshold structure. Panel (b) is a zoomable engineering view of the (p, q) operating region. Panel (c) answers the resource question: how many physical bits are needed to reach a target logical error rate for a given hardware error and cycle count? This separation prevents the generous $p = 0.5$ information boundary from obscuring the much smaller region where useful logical-error targets are achieved.

Simulator Interpretation

Figure 1 summarizes the three simulator panels.

Panel (a) shows the broad majority-vote behavior for several code lengths N at fixed measurement error q and cycle count k . For $p < 0.5$, each bit is more likely correct than wrong, so increasing N can suppress logical failure. The currently selected N is highlighted with a thicker line. At $p = 0.5$ the decoder has no information; for $p > 0.5$ majority voting is systematically misleading. Increasing k raises every curve uniformly, illustrating that sustained operation demands a lower per-cycle error rate.

Panel (b) is the useful operating-region heatmap for fixed N and k . The horizontal axis is hardware error p , the vertical axis is measurement error q , and the color indicates logical failure probability p_L . Red contours mark user-selected logical-error targets. The black dashed contour marks $p_L = p$, where encoding stops improving over a single physical bit.

Panel (c) is the resource-estimation view. Each line shows how the analytical logical failure rate changes with code length N for a fixed hardware error value p at the selected q and k . The currently selected p is drawn as a solid line; others are dashed. Horizontal dotted lines mark the logical-error targets. Reading off the crossing points gives the

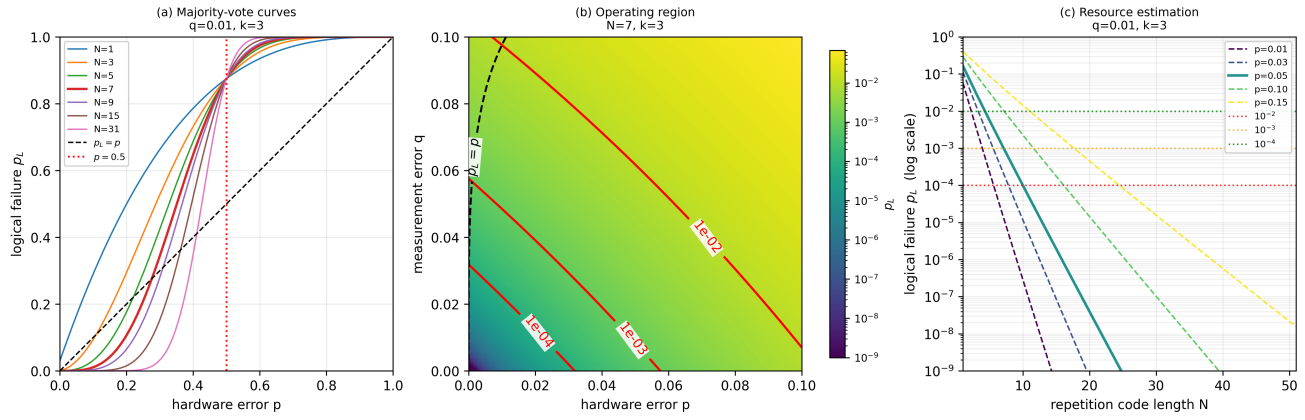


Figure 1: Fault-tolerance scaffold simulator (v2.0). (a) Majority-vote logical failure curves for several repetition lengths N at fixed measurement error q and cycle count k . The dashed diagonal $p_L = p$ is the unencoded reference; the vertical line at $p = 0.5$ marks the total-randomness boundary. The selected N is highlighted. (b) Zoomable hardware–measurement operating region for fixed N and k . Color indicates logical failure probability p_L ; red contours mark user-selected logical-error targets; the black dashed contour marks $p_L = p$. (c) Resource-estimation curve: analytical logical failure rate versus code length N for several hardware error values p at the selected q and k . The selected p is solid; others are dashed. Horizontal dotted lines mark logical-error targets. Crossing points give the minimum N needed to reach each target.

minimum N required to reach each target — the physical-bit overhead implied by the hardware error rate and cycle budget.

Connection to Quantum Error Correction

The simulator is intentionally classical, but the analogy to quantum error correction is direct:

Classical scaffold	Quantum error-correction analogue
Physical bit flip	Physical qubit error
Readout error q	Noisy syndrome measurement
Majority vote	Decoder / inference algorithm
Corrected logical bit	Logical qubit / Pauli-frame update
Logical failure p_L	Logical error per round or operation
Cycle count k	Circuit depth / number of QEC rounds
Resource curve N vs p_L	Physical-to-logical qubit overhead

The important difference is that quantum information cannot simply be copied and directly measured. Quantum codes use syndrome measurements to infer error information without directly measuring and destroying the protected logical state. Stabilizer-code theory provides the formal framework for this process [2]. Surface-code architectures extend this idea into a hardware-aware layout where local measurements, repeated syndrome extraction, and classical decoding jointly determine logical reliability [3].

Modern demonstrations and architecture proposals now emphasize not only code distance, but also real-time decoding, correlated error events, compiler/runtime integration, micro-architecture, and operation beyond break-even [4, 5, 8, 9]. Thus, the simulator should be read as a scaffold, not a replacement for a quantum code. It teaches the first systems principle: reliable logical behavior emerges from physical redundancy, diagnostic information, software decoding, and a cycle budget that the hardware error rate can sustain.

Takeaway

The simulator makes fault tolerance accessible by separating the architecture into three layers:

- **Hardware layer:** physical bits fail with probability p per cycle.
- **Measurement layer:** the software observes imperfect evidence with error probability q per cycle.
- **Software layer:** a decoder converts noisy evidence into a logical decision and corrective action.

The point $p = 0.5$ is not a practical performance goal; it is the total-randomness boundary where majority vote loses information. Practical fault tolerance is governed by much stricter logical-error targets, and those targets must be maintained across every correction cycle in the computation. The resource-estimation panel translates this constraint into a concrete answer: for a given hardware error rate p and cycle budget k , it shows the minimum code length N required to stay within budget.

This distinction is the key lesson: fault tolerance is a hardware–software co-design problem, and useful operation depends on the combined performance of hardware errors, measurement fidelity, decoder logic, cycle count, and application-level error requirements.

Acknowledgment This material was developed and/or adapted with support from the National Science Foundation through the QCAP-Pilot and QCAP-Design efforts under NSF Award Nos. OSI-2410813 and OSI-2531569. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

References

- [1] D. Aharonov and M. Ben-Or, *Fault-Tolerant Quantum Computation With Constant Error Rate*, arXiv:quant-ph/9906129, 1999.
- [2] D. Gottesman, *Stabilizer Codes and Quantum Error Correction*, Ph.D. thesis, California Institute of Technology, arXiv:quant-ph/9705052, 1997.
- [3] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, *Surface codes: Towards practical large-scale quantum computation*, Physical Review A **86**, 032324, 2012.
- [4] N. Ofek et al., *Extending the lifetime of a quantum bit with error correction in superconducting circuits*, Nature **536**, 441–445, 2016.
- [5] V. V. Sivak et al., *Real-time quantum error correction beyond break-even*, Nature **616**, 50–55, 2023.
- [6] Z. Chen et al., *Exponential suppression of bit or phase errors with cyclic error correction*, Nature **595**, 383–387, 2021.
- [7] R. Acharya et al., *Suppressing quantum errors by scaling a surface code logical qubit*, Nature **614**, 676–681, 2023.
- [8] R. Acharya et al., *Quantum error correction below the surface code threshold*, Nature **638**, 920–926, 2025.
- [9] F. Tripier et al., *Fault-Tolerant Quantum Computing with Trapped Ions: The Walking Cat Architecture*, arXiv:2604.19481, 2026.
- [10] IBM Quantum, *How IBM will build the world’s first large-scale, fault-tolerant quantum computer*, IBM Quantum Blog, 2025.
- [11] Riverlane, *Riverlane publishes QEC Technology Roadmap*, Riverlane Press Release, 2024.
- [12] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 10th Anniversary Edition, 2010.